

std::deque

```
#include <deque>
```

C++ STL Double-Ended Queue

What is std::deque?

Double-Ended Queue

A sequence container that supports fast insertion and removal at both the front and the back, with random access to elements in $O(1)$ amortised time.

$O(1)$

push_front / push_back

$O(1)$

Random Access []

$O(n)$

Insert / Erase (middle)

Memory Layout (Segmented Buffer Model)



← Each chunk is a fixed-size contiguous array, connected via a central map (pointer table)

Basic Usage

```
#include <deque>
#include <iostream>

int main() {
    std::deque<int> dq;

    // Push to both ends
    dq.push_back(10);    // [10]
    dq.push_back(20);    // [10, 20]
    dq.push_front(5);    // [5, 10, 20]
    dq.push_front(1);    // [1, 5, 10, 20]

    // Random access
    std::cout << dq[0];    // 1
    std::cout << dq.at(2);  // 10
    std::cout << dq.front(); // 1
    std::cout << dq.back();  // 20

    // Pop from both ends
    dq.pop_front();    // [5, 10, 20]
    dq.pop_back();     // [5, 10]

    // Iteration
    for (auto& v : dq) std::cout << v << " ";
}
```

Modifiers

- push_front / push_back
- pop_front / pop_back
- insert / erase
- emplace_front / back
- resize / clear

Access

- operator[] — no bounds check
- at() — bounds-checked
- front() / back()
- begin / end iterators
- size() / empty()

How the STL Designs `std::deque`

Segmented Memory

- Fixed-size chunks (e.g. 512 bytes)
- Allocated on demand
- No single large allocation
- Cache-friendly within each chunk

Central Map (T^{**})

- Array of pointers to chunks
- Grows at both ends
- Allows $O(1)$ subscript via arithmetic
- Map re-allocated if exhausted

Iterators

- 4-pointer iterator: `cur`, `first`, `last`, `node`
- Knows chunk boundaries
- Supports random access arithmetic
- Invalidated on `push_front/back` (map resize)

Chunk Layout & Map Pointer Table



std::deque — Architecture Overview

Part 1: The Three Core Components

std::deque stores data in fixed-size chunks (~512 bytes each). A central pointer map tracks all chunks. Three objects coordinate everything:

start

curr → → first element of deque
first → → start of first data block
last → → end of first data block
node → → location of first data block

finish

curr → → last element of deque
first → → start of last data block
last → → end of last data block
node → → location of last data block

data_map

Array of pointers to all data blocks. Min size: 8 (from stdlib). Grows as needed.

Does NOT hold actual data — only references so blocks can be found.

Initial state: deque<int> dq = {10, 20, 30, 40}



push_back(50) — Insert at the Back

finish.curr increments; element assigned
in place

Since there is empty space in dblock1, finish.curr simply moves right by one and 50 is assigned there. No new block needed.

BEFORE



push_back(50)
↓

AFTER



What happens inside

- ① Check: `finish.curr != finish.last`
→ TRUE: empty slot exists
- ② Assign value at finish.curr:
`*finish.curr = 50`
- ③ Advance the pointer:
`++finish.curr`

No allocation. No new block.

Pure $O(1)$ — just a pointer move.

push_front(0) — New Block Allocated at Front

start.curr can't go left → deque creates
dblock2

start.curr tries to decrement. But it's already at start.first — the left edge of dblock1. Going further would be invalid. A new block is created.

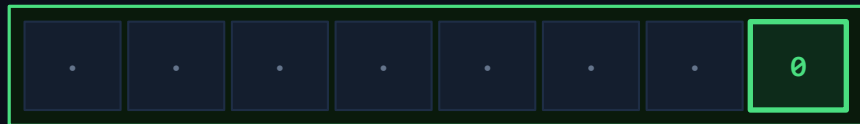
BEFORE push_front(0)

start.curr == start.first → no room to the left!



push_front(0) → allocate dblock2

AFTER — dblock2 created to the left



↑ start.curr → last slot of dblock2

Why the LAST slot?

emplace_front places the new element by first decrementing start.curr.

With dblock2 freshly allocated, start.curr starts at start.last (one-past-end), then --start.curr lands on the last valid slot.

Next push_front will fill slots right-to-left: [6], [5], [4]...

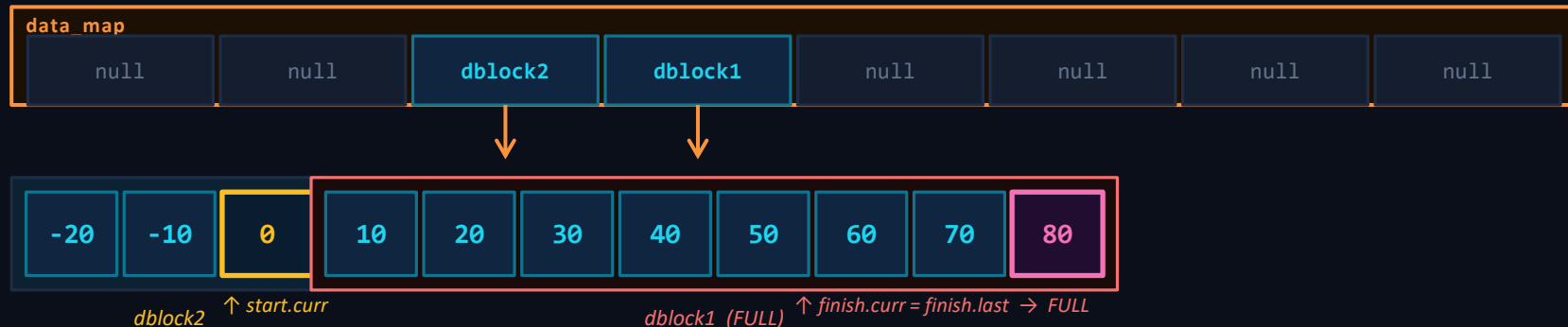
This mirrors how emplace_back fills slots left-to-right.

push_back(60,70,80) then push_back(90) — Filling a Block

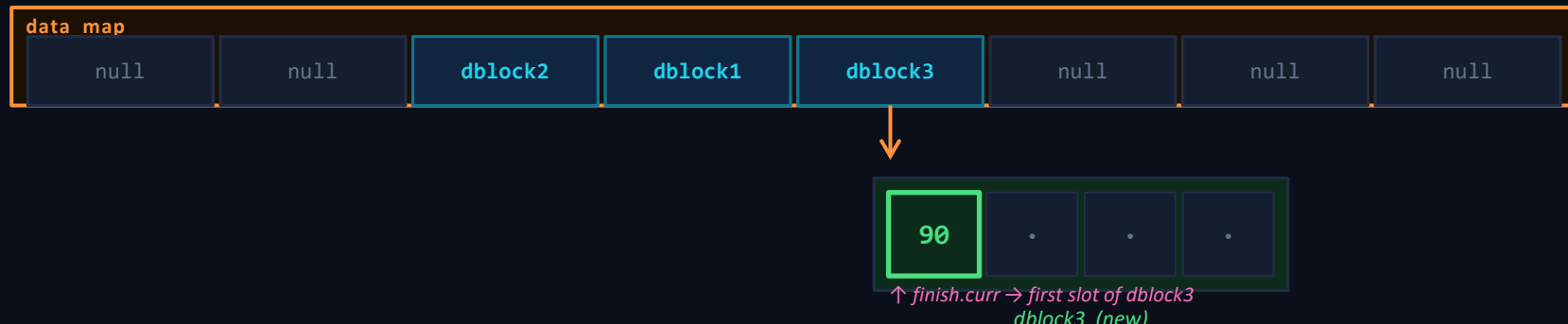
When the last block is full, a new one (dblock3) is allocated

Three push_back calls fill the remaining slots in dblock1. The fourth (90) finds the block full — a new dblock3 is created and 90 goes at its first slot.

After push_back(60), push_back(70), push_back(80) — dblock1 is now FULL



push_back(90) → finish.curr == finish.last → allocate dblock3

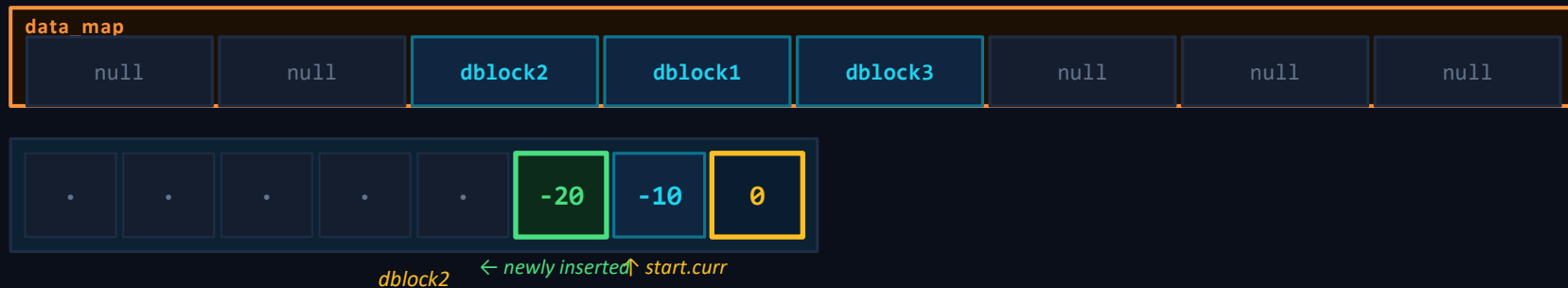


push_front(-10, -20) then pop_back()

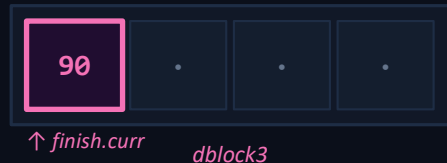
Front fills right-to-left; deletion just moves the pointer

dblock2 still has empty slots to the left of -10. push_front simply decrements start.curr twice. pop_back on a single-element block deallocates the whole block.

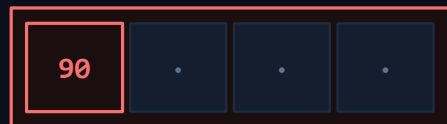
After push_front(-10) and push_front(-20)



dblock1: [10,20,30,40,50,60,70,80] (unchanged, full)



pop_back() → dblock3 has only one element → entire block freed



finish now points back to dblock1.
finish.curr → slot of element 80.

Lazy deletion: element is NOT immediately erased — curr just moves. Only when a full block is empty does deque free it.

The Iterator & data_map Internals

How navigation and growth work under the hood

`_Deque_iterator` struct

```
struct _Deque_iterator {  
    T* curr;    // current element  
    T* first;   // chunk start  
    T* last;    // chunk end (past)  
    T** node;   // slot in data_map
```

```
self& operator++() {  
    ++curr;  
    if (curr == last) {  
        // crossed chunk boundary!  
        set_node(node + 1);  
        curr = first;  
    }  
    return *this;  
}
```

```
void set_node(T** new_node) {  
    node = new_node;
```

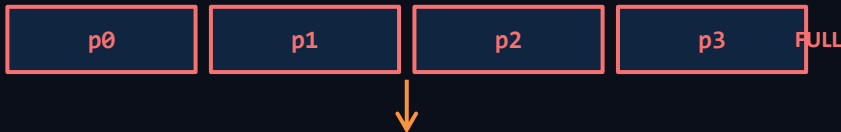
Key insight: references vs iterators

push/emplace at either end never moves existing elements. Chunks stay fixed in memory — raw T* pointers remain valid. Only the iterator's node field (its slot in data_map) may be stale after a map realloc.

Safe pattern: store indices, not iterators, across mutations.

data_map growth — when map is full:

Old (full):



2× realloc, ptrs copied to center:



Iterator invalidation rules:

Operation	Iterators	References
push_back / emplace_back	⚠ if map realloc	✅ stable
push_front / emplace_front	⚠ if map realloc	✅ stable
pop_back / pop_front	⚠ if map shrinks	✅ stable*
insert / erase (middle)	❌ all invalid	❌ invalid

* erased element's ref invalid; others stable

Iterators, Invalidation & Advanced Usage

```
// Iterator invalidation rules
std::deque<int> dq = {1, 2, 3, 4, 5};

auto it = dq.begin() + 2; // points to 3

dq.push_back(6); // all iterators invalidated
                // if map realloc occurs

dq.push_front(0); // all iterators invalidated

// Insert in middle
dq.insert(dq.begin() + 1, 99); // invalidates ALL

// Safe pattern: use indices, not iterators
size_t idx = 2;
dq.push_back(7);
auto safe = dq.begin() + idx; // re-derive after push

// Rotate / slice with deque
std::rotate(dq.begin(), dq.begin()+1, dq.end());

// Use as sliding window
while (dq.size() > 5) dq.pop_front();
```

⚠ Invalidation Rules

- `push_back` / `push_front`: invalidates ALL iterators if the map pointer array is reallocated
- `insert` / `erase` in middle: ALL iterators invalidated
- `insert` at front/back: only references, not necessarily pointers
- Always re-derive iterators after mutation

✅ Best Practices

- Use index arithmetic after bulk push
- Prefer range-for for iteration
- Use `shrink_to_fit()` to release memory
- Avoid repeated middle inserts (use list)

std::deque as a choice



Order Book Management (MEOrdersAtPrice doubly linked list)

- Bid/ask levels stored as deque of price-qty pairs
- New orders pushed to front or back by price
- $O(1)$ best-price access from front/back
- Cancels erase from middle — acceptable cost



Time-Series VWAP Window

- VWAP computed over rolling N-tick window
- push_back new trade; pop_front stale trade
- No reallocation overhead mid-session
- Deque preserves insertion order naturally

Trading System: Order Book Management

```
struct MEOrdersAtPrice {
    Side      side_;
    Price     price_;
    MEOrder*  first_me_order_; // ← head of FIFO order chain

    MEOrdersAtPrice* prev_entry_; // ← links to adjacent price levels
    MEOrdersAtPrice* next_entry_; // ← in the sorted list
};

struct MEOrdersAtPrice_Deque {
    Side      side_;
    Price     price_;
    std::deque<MEOrder*> orders; // replaces the MEOrder linked list
                                // and first_me_order_ pointer

    // prev_entry_ / next_entry_ still needed for the price level list
    MEOrdersAtPrice_Deque* prev_entry_;
    MEOrdersAtPrice_Deque* next_entry_;
};
```

Book approach (intrusive linked list + MemPool)

Zero heap during trading

All objects from MemPool pre-allocated at startup. No malloc, no free, no heap lock contention during market hours.

O(1) cancel, stable ptrs

L1 gives MEOrder* directly. Intrusive prev_/next_ let you unlink in O(1). MemPool never moves objects — pointer stable forever.

Deterministic latency

Every operation touches a bounded number of cache lines. No hidden allocator calls. P99/P999 stay tight — critical for exchange infra.

Disadvantages

Cache-unfriendly iteration

Walking FIFO chain follows MEOrder* pointers scattered across MemPool. Each hop may miss L1/L2 cache. Deque prefetches better.

Intrusive coupling

MEOrder carries prev_/next_ — 16 bytes of list machinery baked into the domain struct. Harder to reuse outside this data structure.

Manual pointer management

Every add, cancel and fill requires updating 4-6 pointers by hand. One wrong write corrupts the list silently with no safety net.

std::deque alternative

Simpler code

push_back, pop_front, front() — standard API. No manual pointer wiring. Much harder to introduce a corruption bug.

O(n) cancel scan

To cancel by order id you must scan the deque for the MEOrder*. std::deque has no O(1) middle removal.

Better cache on iteration

MEOrder* stored in contiguous deque chunks. Iterating all orders at a level is sequential — CPU prefetcher works well.

Iterator invalidation risk

push_back/front can invalidate all iterators on map realloc. Cached raw MEOrder* stay safe, but deque iterators cannot be held across mutations.

Heap alloc on level create

Every new price level triggers malloc (deque ctor). Every empty level triggers free. Constant heap pressure — non-deterministic latency.

Custom allocator can help

std::deque<MEOrder*, PoolAlloc> eliminates heap pressure. But you end up writing most of the same infrastructure anyway.

Verdict

Exchange matching engine — use the book's approach. Cancel is the most frequent op in a volatile market: O(1) vs O(n) matters enormously. Zero heap allocs eliminate P99 latency spikes.

Buy-side / prototype — std::deque is fine. Simpler code, cancel events are rarer, and a custom pool allocator recovers most of the latency profile. Deque genuinely wins only on cache-friendly sequential iteration.

Trading System: VWAP Sliding Window

```
struct Trade { double price; long qty; };

class VWAPWindow {
    std::deque<Trade> window_;
    size_t max_ticks_;
    double running_pv_ = 0;    // price * volume
    long    running_vol_ = 0;

public:
    explicit VWAPWindow(size_t n) : max_ticks_(n) {}

    void on_trade(double px, long qty) {
        // Add new trade - O(1) amortised
        window_.push_back({px, qty});
        running_pv_ += px * qty;
        running_vol_ += qty;

        // Evict stale trades from front - O(1)
        while (window_.size() > max_ticks_) {
            auto& old = window_.front();
            running_pv_ -= old.price * old.qty;
            running_vol_ -= old.qty;
            window_.pop_front();
        }

        double vwap() const {
            return running_vol_ ? running_pv_ / running_vol_ : 0.0;
        }
    };
};
```

O(1) push_back
no realloc

O(1) pop_front
evicts stale data

Why deque wins here:

- ✓ No reallocation pressure
- ✓ Amortised O(1) both ends
- ✓ Custom allocator slots in
- ✓ Index access for snapshots

Advantages & Disadvantages vs Other STL Containers

Container	push_front	push_back	Random Access	Memory	Iterator Safety	Best For
std::deque	O(1)	O(1)	O(1)	Segmented	Fragile	Queue / sliding window
std::vector	O(n) ⚠	O(1)*	O(1)	Contiguous	On realloc	General sequence / search
std::array	N/A	N/A	O(1)	Stack alloc	Stable	Fixed-size, stack buffers
std::list	O(1)	O(1)	O(n) ⚠	Node heap	Stable	Frequent middle inserts
std::forward_list	O(1)	O(n) ⚠	O(n) ⚠	Node heap	Stable	Minimal-overhead singly linked

✅ std::deque highlighted | ⚠ = performance concern | * = amortised | Stable = no invalidation on push_front/push_back (unlike deque)

Deep Dive: deque vs vector vs array

`std::vector`

-  Single contiguous block → best cache locality
-  Fastest random access and iteration in practice
-  `push_back` amortised $O(1)$; reserves capacity upfront
-  `push_front` is $O(n)$ — shifts all elements
-  Reallocation invalidates ALL iterators & pointers
-  Large allocations can cause heap fragmentation
- → Use when: append-only, random-access heavy, no front ops

`std::array<T, N>`

-  Zero heap allocation — lives on the stack
-  Best for ultra-low-latency fixed-size buffers
-  No iterator invalidation, size known at compile time
-  Fixed size — cannot grow or shrink at runtime
-  No `push_front` / `push_back` API
-  Large N overflows the stack — use with caution
- → Use when: fixed-size ring buffers, lookup tables, HFT micro-structures

`std::deque` — The Sweet Spot

-  $O(1)$ push/pop at both ends — no $O(n)$ shifts
-  No single massive reallocation — segmented growth is gentler on the allocator
-  $O(1)$ random access (with minor indirection vs vector)
-  Slightly slower iteration than vector due to chunk-boundary jumps
-  Higher memory overhead: pointer map + chunk headers
-  Iterator invalidation rules are subtle — fragile under mutations
- → Use when: sliding windows, bounded queues, order books, event buffers

Summary: When to Choose `std::deque`

✓ Use `std::deque` when...

- Need $O(1)$ push/pop at both ends
- Sliding window over streaming data
- Order book price levels
- Event replay with index access
- Custom allocator via `Allocator` param

🔄 Prefer `std::vector` when...

- Append-only workload
- Cache-critical tight loops
- Single large dataset in memory
- STL algorithms (sort, `binary_search`)
- No front insertions needed

⚡ Prefer `std::array` when...

- Compile-time known size
- Zero-allocation requirement
- Stack-allocated ring/lookup buffer
- Ultra-latency critical path (HFT)
- Embedded / `constexpr` contexts

Key Takeaway

`std::deque` is the go-to container when you need random access AND efficient operations at both ends — the sweet spot between `std::vector`'s cache performance and `std::list`'s flexible insertion. In low-latency trading systems, its segmented allocation, $O(1)$ endpoints, and custom-allocator support make it the backbone of market-data pipelines and order book engines.